

Agent

ShockBench-Flow – Main Agent Algorithm

1. Goal

Our goal is to create a **self-contained autonomous logistics agent** that minimizes the total economic cost of operating the supply network under uncertainty.

The agent does not know future disruptions.

Every simulated week it receives the current state of the network and must decide:

> **How much of each good should be sent through each available route this week?**

The main optimization objective is to minimize:

- freight costs;
- tariffs;
- storage costs;
- backlog / unmet demand costs;
- losses caused by delays and disruptions.

2. Main Agent Cycle

The main decision-making process is:

```
```text
Observation
 ↓
Current State Analysis
 ↓
Inventory Forecast
 ↓
Shortage Detection
 ↓
Early Warning Analysis
 ↓
Risk Estimation
```

↓  
Target Inventory Calculation  
↓  
Route Evaluation  
↓  
Flow Allocation  
↓  
Action

This process is repeated every simulated week.

---

### 3. Agent Interface

The submission contains an `Agent` class.

```
class Agent: def __init__(self, config): ... def act(self, observation): ... return {"flows": flows}
```

#### `__init__(self, config)`

Called **once at the beginning of an episode**.

Its responsibilities:

1. Read the network configuration.
2. Determine the number of routes/action slots.
3. Store static route information.
4. Store normal route capacities.
5. Store network structure.
6. Initialize agent memory.
7. Initialize policy parameters.

Example internal state:

```
self.week = 0self.warning_history = []self.stock_history = []self.cost_history = []self.safety_factor = ...self.risk_weight = ...
```

Network sizes must never be hard-coded.

The agent must read all required shapes and dimensions from `config`.

---

## 4. Weekly `act()` Algorithm

Every week the agent receives a new `observation` .

The method should execute the following logical stages.

---

### Stage 1 — Read Current State

Extract the important information from the observation.

Main inputs:

- `stock`;
- `backlog`;
- `goods in transit`;
- `queued goods`;
- `demand forecast`;
- `route capacities`;
- `route costs`;
- `lead times`;
- `tariffs`;
- `sanctions`;
- `strait openness`;
- `Early Warnings`;
- `announcements`;
- `action_mask` .

Conceptually:

```
What do we currently have?
What is already travelling?
What demand has not been satisfied?
Which routes are currently usable?
```

---

## 5. Update Agent Memory

The agent should keep information from previous weeks.

Example:

```
warning history
stock history
previous costs
previous flows
previous backlog
```

This allows the agent to distinguish between:

```
warning = 0.7
```

and:

```
0.1 → 0.3 → 0.5 → 0.7
```

The second case may indicate a rapidly increasing risk.

---

## 6. Forecast Future Inventory

The agent should estimate how inventory may change over the next several weeks.

Basic idea:

```
projected inventory
=
current stock
+ expected arrivals
- expected demand
```

Example:

```
Current stock = 100
Incoming goods = 30

Forecast demand:
Week +1 = 20
Week +2 = 25
Week +3 = 30
```

```
Week +4 = 35
```

```
Total demand = 110
```

```
Projected inventory:
```

```
100 + 30 - 110 = 20
```

The agent should also consider backlog.

Conceptually:

```
effective demand
=
forecast demand
+ existing backlog
```

---

## 7. Detect Future Shortages

The agent should not wait until inventory reaches zero.

It should detect shortages **before they happen**.

Example:

```
Week +1: stock = 80
Week +2: stock = 55
Week +3: stock = 20
Week +4: stock = -15
```

The agent can see that a shortage is likely in Week +4.

If the required route has:

```
lead time = 3 weeks
```

then the shipment must potentially be made now.

---

## 8. Analyse Early Warnings

Early Warning is a signal that a future disruption **may** happen.

It is not certainty.

Therefore:

```
Early Warning ≠ guaranteed disruption
```

Bad behaviour:

```
warning appears
→ completely stop using the route
```

Preferred behaviour:

```
warning increases
→ risk increases gradually
→ agent becomes gradually more defensive
```

Possible reactions:

- increase safety stock;
- reduce dependence on a risky route;
- use alternative routes;
- diversify shipments;
- send important goods earlier.

---

## 9. Calculate Risk

The agent may convert available signals into an internal `risk_score` .

Example conceptual model:

```
risk =
warning component
+ warning trend
+ current congestion
+ current disruption state
+ sanctions component
```

Example interpretation:

Risk 0.0–0.2

→ normal operation

Risk 0.2–0.5

→ slightly larger buffers

Risk 0.5–0.8

→ increased inventory + diversification

Risk 0.8–1.0

→ strong defensive routing

The exact thresholds must be determined experimentally.

---

## 10. Calculate Target Inventory

The agent should determine how much inventory it wants to maintain.

Basic concept:

target inventory

=

expected future demand

+ safety stock

Safety stock may depend on risk.

Example:

Low risk:

3 weeks of demand

Medium risk:

4 weeks of demand

High risk:

5 weeks of demand

Conceptually:

Early Warning ↑



Risk ↑



Safety Stock ↑

But excessive inventory must be avoided because storage is expensive.

---

## 11. Calculate Required Shipment

After calculating target inventory:

```
required shipment
=
target inventory
- current stock
- incoming goods
```

If:

```
required shipment <= 0
```

the agent may not need additional shipments.

If:

```
required shipment > 0
```

the agent must determine how to deliver that amount.

---

## 12. Evaluate Routes

Each available route should be evaluated economically.

Conceptual route score:

```
route score
=
```

```
freight cost
+ tariff penalty
+ lead-time penalty
+ disruption risk penalty
+ congestion penalty
```

Lower score means a more attractive route.

Example:

Route	Cost	Lead Time	Risk
A	10	2 weeks	High
B	14	3 weeks	Low
C	18	1 week	Low

During normal conditions, Route A may be best.

During a serious Early Warning, Route B may become economically preferable.

For urgent shipments, Route C may become useful because of its short lead time.

---

## 13. Diversification

The agent should not necessarily send everything through one route.

Example:

```
Normal:
Route A → 90%
Route B → 10%

Moderate risk:
Route A → 65%
Route B → 35%

High risk:
Route A → 30%
Route B → 70%
```

Diversification reduces dependence on a single chokepoint.

However, unnecessary diversification may increase costs.

Therefore it should be risk-dependent.

---

## 14. Allocate Flows

After determining:

- required shipment;
- route scores;
- route capacities;
- route availability;

the agent converts the plan into actual `flows` .

Example:

```
Need = 100 units
```

```
Route A:
capacity = 40
```

```
Route B:
capacity = 80
```

```
Decision:
```

```
Route A → 30
Route B → 70
```

The allocation must respect route capacity and other constraints.

---

## 15. Apply Action Mask

Before returning the action, the agent must respect `action_mask` .

Conceptually:

```
flows = flows * observation["action_mask"]
```

Unavailable routes must not receive shipments.

---

## 16. Return Action

Finally:

```
return { "flows": flows}
```

Optional tanker queue controls may be added later:

```
release_mode
override_qty
```

The first version of the agent should focus primarily on `flows` .

---

## 17. Core Decision Formula

The central decision of the agent can be expressed as:

**Is the additional cost of protecting the network now lower than the expected cost of doing nothing and suffering a disruption later?**

Example:

```
Normal route cost:
$10M
```

```
Safer route cost:
$14M
```

```
Additional protection cost:
$4M
```

If a potential disruption could create:

```
$40M expected shortage loss
```

then paying the additional `$4M` may be economically rational.

---

## 18. Main Economic Trade-Off

The agent continuously balances:

CHEAPER NOW  
vs  
SAFER LATER

Too aggressive:

large buffers  
many alternative routes  
high storage  
high freight cost

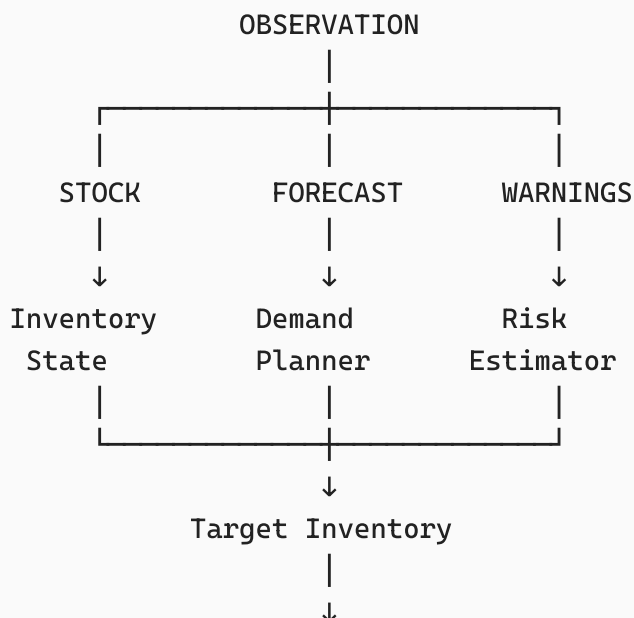
Too passive:

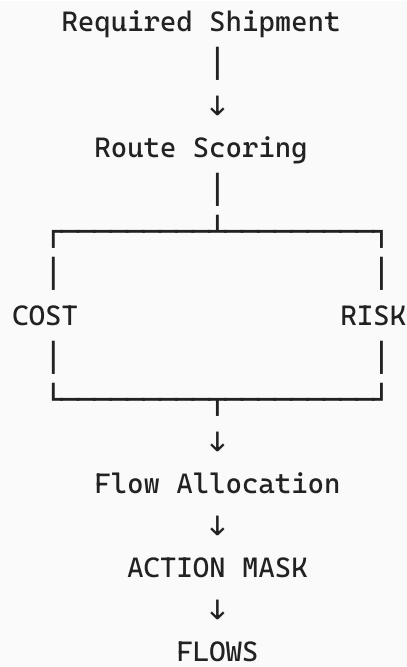
low costs initially  
but large backlog during disruptions

The best policy should find the lowest total expected cost.

---

## 19. Proposed Agent Architecture





---

## 20. Suggested Internal Methods

A clean implementation may look like:

```
class Agent:
 def __init__(self, config):
 ...
 def update_memory(self, observation):
 ...
 def estimate_risk(self, observation):
 ...
 def forecast_inventory(self, observation):
 ...
 def calculate_needs(self, observation, risk):
 ...
 def score_routes(self, observation, risk):
 ...
 def allocate_flows(self, observation, needs, route_scores):
 ...
 def act(self, observation):
 ...
```

Main `act()` structure:

```
def act(self, observation):
 self.update_memory(observation)
 risk = self.estimate_risk(observation)
 projected_inventory = self.forecast_inventory(observation)
 needs = self.calculate_needs(observation, risk)
 route_scores = self.score_routes(observation, risk)
 flows = self.allocate_flows(observation, needs, route_scores)
 flows *= observation["action_mask"]
 return {"flows": flows}
```

---

## 21. Development Algorithm

The agent itself should be developed incrementally.

```
Baseline
 ↓
Inventory-aware
 ↓
Forecast-aware
 ↓
Early-Warning-aware
 ↓
Risk-aware routing
 ↓
Diversification
 ↓
Parameter optimisation
```

Do not add everything at once.

---

## 22. Experiment Workflow

For every new idea:

```
HYPOTHESIS
 ↓
IMPLEMENT
 ↓
TEST
 ↓
SBF COMPARE
 ↓
┌───┴───┐
│ │
BETTER WORSE
│ │
KEEP REJECT
```

Example:

Hypothesis:

Increasing safety stock when Early Warning rises will reduce disruption backlog more than it increases freight and storage costs.

Then compare:

```
sbf compare new_agent old_agent --task=small
```

---

## 23. Main Rule

Every change to the policy should answer:

1. What problem are we trying to solve?
2. Which observation fields are used?
3. What behaviour changes?
4. Why should this reduce total cost?
5. How will we test the hypothesis?
6. Did `sbf compare` confirm the improvement?

---

## 24. Final Mental Model

Every week the agent asks five questions:

1. WHAT DO I HAVE?  
stock + incoming + backlog
2. WHAT WILL I NEED?  
demand forecast
3. WHAT CAN GO WRONG?  
Early Warnings + disruptions
4. HOW CAN I DELIVER IT?  
routes + capacity + lead time
5. WHAT IS ECONOMICALLY BEST?  
cost + risk + future consequences

Then:

choose flows

The goal is not to predict the future perfectly.

The goal is:

**Make economically optimal logistics decisions given incomplete information and uncertainty.**